INF-1400: Object-oriented programming

Assignment 2

Jens Christian Valen Leynse

March 8th, 2024

1 Introduction

The assignment was to create a simple simulator clone of the classic flock simulator boids. The code is written in python 3 using an object-oriented design including classes methods and the requirements for the assignment.

1.1 Requirements

1. The flock of boids should be able to move in a lifelike manner without hardcoding the movements.
2. Implement the simulator in accordance with object-oriented design, using objects and classes.
3. Inheritance must be used to implement at least one class.
4. simulator must follow the rules described above.
5. Hoiks and obstacles must be implemented.
6. The report must give a description of inheritance in object-oriented programming, and how you have chosen to use this feature.
7. The hand-in must include a class diagram (as shown in lectures) which describes relations between the different classes

1.2 The boids should follow set rules in their movements.

1. Boids steer towards the average position of local flockmates.
2. Boids attempt to avoid crashing into other boids.
3. Boids attempt to avoid crashing into other boids.
4. Obstacles the boids need to avoid.
5. Predators (hoiks) that will try to eat the boids.

Extra:

6. Hoiks can eat boids and gain size.
7. Bait that attract boids.
8. Use the mouse to add boids, obstacles, bait or hoiks

2 Technical Background

The code utilizes the pygame library to set up modules that feature various display options: setting up the screen, updating it, and making changes visible. It includes modules for handling time, enabling delay between actions, playing music to add ambiance to the game, tracking the mouse for input, managing events including mouse events, graphic events, and exiting events, drawing objects such as geometric figures, handling images by loading and rendering them, and facilitating graphic user interface interactions by creating visible objects for user interactions. Additionally, the code imports math and random libraries to calculate more intricate values. Moreover, the code incorporates object-oriented programming features like classes, subclasses, mixins, and inheritance, along with encapsulation, polymorphism, composition, and abstraction.

3 Design

The objects module in the boids simulator comprises four main classes: boids, hoiks, obstacles, and bait. These classes represent individual entities within the simulation, each possessing its own set of attributes, including position, velocity, and orientation. Boids and hoiks are defined with attributes for position and velocity, enabling interactions with other objects based on predefined rules. Obstacles require only a position for visual representation, while bait, similarly needs the position to be manually moved via user input. The FlyerList class acts as a container, efficiently managing multiple instances of these objects while promoting encapsulation and modularization.

The Rulebook module facilitates the simulation of complex emergent behaviors such as flocking, hunting, collision avoidance, and target prioritization. Within the Rulebook class, methods are implemented to centralize entities, match vectors, avoid collisions, evade predators, prioritize targets, and explore surroundings. These functionalities enable accurate replication of real-world behaviors within the simulation environment.

The behavior module in the boids simulator consists of three main classes: Drawing, Movement, and Behavior, each encapsulating specific functionalities related to the objects' behavior and appearance. The Drawing class handles visual representation on the screen, providing methods to draw polygons and circles with customizable attributes. Movement manages the dynamics of boids and hoiks, incorporating rules from the Rulebook class. Lastly, the Behavior class orchestrates additional behaviors such as grouping, growth and utilizing information from the environment to drive decision-making. This modular design promotes easy addition or modification of behaviors without affecting other parts of the system.

The GUI module of the boids simulator creates and manages graphical user interface elements to control various simulation aspects. It initializes a panel on the right side of the screen, featuring sliders, buttons, and labels for adjusting parameters related to boids, hoiks, and other simulation settings. These elements are dynamically positioned relative to the panel, allowing for flexible resizing and layout adjustment. Leveraging the pygame_gui library, the GUI module provides a user-friendly interface, offering real-time updates as the simulation progresses to give users immediate feedback on their actions and object behaviors.

The game module of the boids simulator oversees the game loop, user input, state updates, and graphics rendering. It interacts with the GUI module to respond to actions controlling simulation aspects. Utilizing the pygame library, the game module creates the game window, manages events, and handles graphics rendering. The main game class orchestrates the simulation within the game loop, continuously updating object states based on predefined behaviors.

The config and resources files serve as auxiliary modules, offering easily adjustable variables and methods for the boids simulator. The config makes it easier for users, even without prior knowledge, to modify parameters and tailor the simulation to their preferences. Conversely, the resources file streamlines object interaction within the simulation environment.

4 Implementation

In the object tmodule, each object class is equipped with attributes defining its position, velocity, and orientation. Specialized methods are tailored for distinct tasks such as obstacle rendering and bait activation. Both boids and hoiks inherit movement functionalities from the Flyer class. The FlyerList container handles object management tasks, including creation with quantity control and removal using lists, while also updating the attributes. Obstacles are rendered uniquely due to their utilization of sprites. Bait objects are restricted to one instance at a time, ensuring controlled activation. Additionally, there are methods to coordinate object movement, behavior, and rendering.

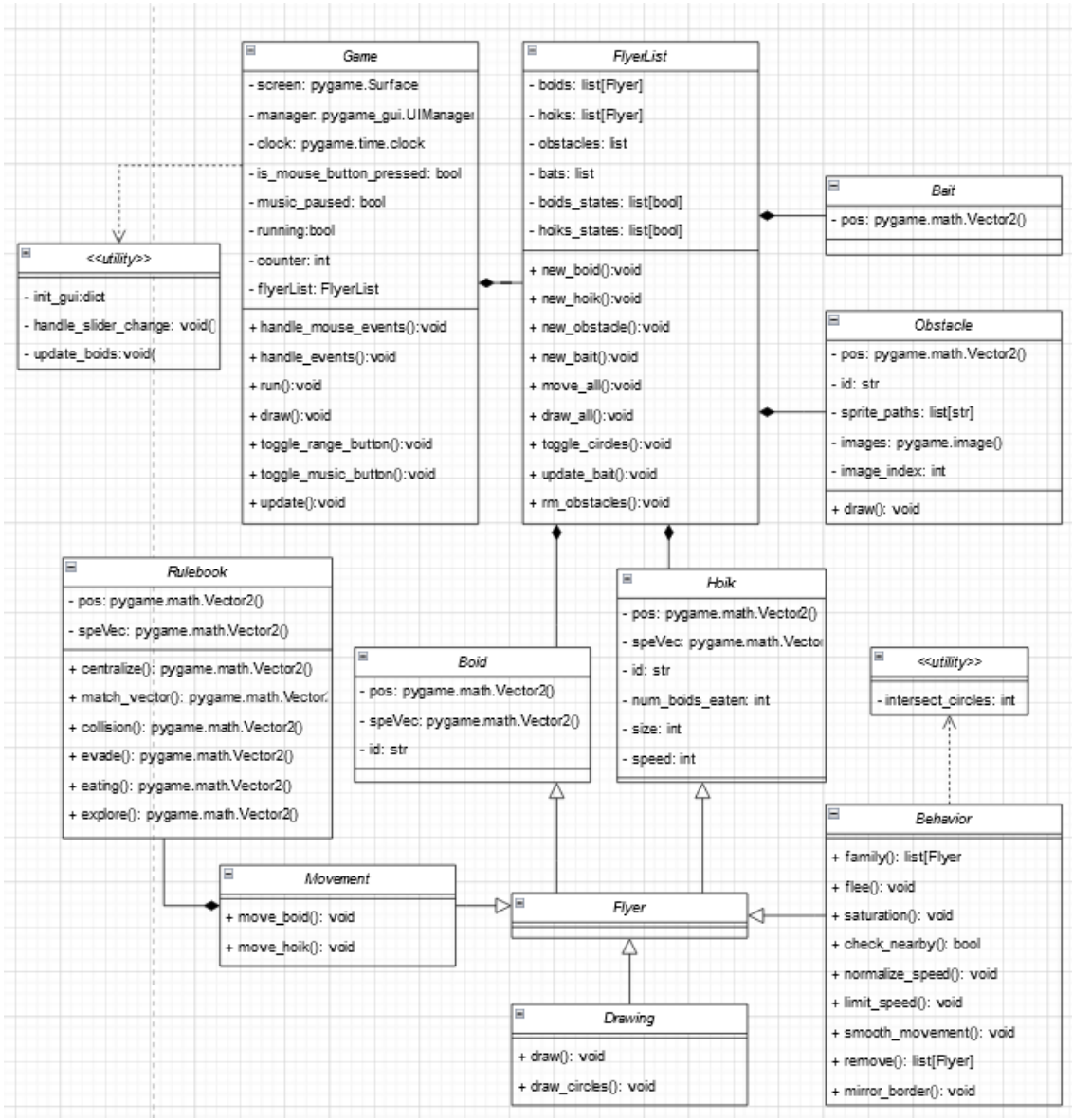
In the rulebook module, a set of movement rules is implemented to transform object interactions and movements, facilitating behaviors such as flocking and collision avoidance. The centralize rule directs objects towards the center position of nearby entities, promoting group cohesion. The match speed rule encourages objects within a group to align their velocities, enhancing coordination. To avoid collisions, the collision rule adjusts object trajectories to steer clear of obstacles and other nearby objects. Additionally, the evade rule prompts boids to flee from nearby hoiks, simulating predator evasion. The eating rule prioritizes targets within range, guiding objects towards them for consumption. Lastly, the explore rule introduces randomness to object movement, simulating exploration behavior.

The Behavior module, comprising mixins within the Flyer class, orchestrates the movement and direction of objects. With six primary definitions and additional methods, it governs object behavior comprehensively. Drawing visually represents objects by drawing triangles based on their position and direction. For boids, the Move Boid method coordinates movement by adjusting speed using the rules from the rulebook and applying priority vectors derived from object lists. Similarly, Move Hoik manages hoik movement with distinct priorities. The Family function organizes boids into groups based on proximity. Upon collision with hoiks, the Remove method eliminates boids, simulating predation. Additionally, Saturation adjusts object speed and size based on consumption. The Flee method prompts evasion from nearby hoiks, and the Check Nearby function assesses object proximity. Moreover, Limit Speed restricts object speed magnitude, and Smooth Movement refines movement dynamics. Lastly, the Mirror Border method confines objects within the screen, ensuring uninterrupted simulation dynamics.

The GUI module encompasses components for user interaction with the interface. These components are arranged and stylized according to manual design specifications, focusing on a user-friendly

experience. Event listeners are implemented to detect user input, triggering corresponding actions within the simulation. The interface elements contribute to control of the simulation parameters. Interactions with these elements are also handled by creation or removal of objects and updating the current objects on the screen.

In the Boids module, the primary game class orchestrates the simulation through a loop. This loop ensures continuous updates to the simulation state. The class also handles user input and GUI interaction. Management classes oversee object existence, managing creation, removal, and organization into lists. Exception handling enforces constraints, maintaining stability. Lastly, the game class adjusts the loop to ensure seamless movement continuity. This setup enables efficient simulation execution and management.



All of the variables utilized in the code are imported from a configuration file.

5 Discussion

Optimization options include addressing sprite handling, as creating new obstacles currently slows down flying objects, and an excess of obstacles affects framerate. Although the "explore" behavior aids movement, it lacks the intended impact, suggesting potential improvements to this rule or its interaction with other rules for enhanced movement.

Corrupted music on my personal computer limited experimentation. Separating the flyer class into three mixins helped manage its overwhelming size while maintaining functionality. Implementing a counter for obstacle creation aimed to avoid creating obstacles every frame, replacing a processor-intensive random number generator from an earlier version.

Initially using mouse inputs for interaction, a controller panel was later adopted for thorough, long-term testing and visualization. Vector calculation weights in the behavior module were based on personal judgment, leaving room for potential improvements. The decision to omit a screen border aimed to challenge hoiks during hunting, prior to the size implementation. Mirrored walls facilitated boid movement, highlighting the centralizing rule.

Attempts to improve turn angles initially resulted in choppy movement due to frame-by-frame calculations. Future improvements could involve relaxing the boid object's strictness on speed vectors, reducing choppy direction changes around obstacles or hoiks.

6 Conclusion

The boids simulation project meets all assignment requirements and combines object-oriented design, behavior modeling, GUI development, game logic, and configuration management. Boids exhibit semi-lifelike behavior, interacting with obstacles and hoiks as intended. With its modular structure, the project ensures maintainability, extensibility, and ease of use, representing effective software engineering principles in interactive simulation development.

References

The pygame library with its many functions included:

https://pygame.org/docs/

A descriptive page on how a boid works:

http://www.red3d.com/cwr/boids/

A pseudocode of the boids movements:

http://www.kfish.org/boids/pseudocode.html

Another description of the rules for the boids (differently explained):

http://code.activestate.com/recipes/502216-boids-demonstration/